# Algorithm Engineering
# Woche 1, Aufgabe 4

Jan Vogt

## Monte Carlo $\pi$ mit OpenMP (Folie 21)

Bei diesem Programm werden zufällige Punkte im Einheitsquadrat erzeugt und es wird gezählt, wie viele davon im Viertelkreis liegen. Das Verhältnis ist ungefähr $\pi/4$. Im Ordner liegen folgende Dateien:

- `pi_monte_carlo.cpp`: die serielle Version von der Folie
- `pi_monte_carlo_parallel.cpp`: meine parallele Version
- `seed_test.cpp`: ein kleiner Test für den Zufallsgenerator

Getestet habe ich wieder auf meinem Laptop (Apple M3, 4 Performance- und 4 Effizienz-Kerne) mit `g++-16 -fopenmp`.

### Das Problem mit dem Zufallsgenerator

Anders als beim $\pi$-Programm mit Integration kann man die Schleife hier nicht einfach aufteilen. Alle Durchläufe benutzen denselben Zufallsgenerator `re`, der bei jedem Aufruf seinen internen Zustand ändert. Würden sich alle Threads diesen Generator teilen, würden sie gleichzeitig in denselben Zustand schreiben. Das wäre ein Data Race und die Threads könnten dadurch auch dieselben Zufallszahlen bekommen.

Deshalb bekommt in meiner Version jeder Thread seinen eigenen Generator mit einem eigenen Seed. Die naheliegende Idee wäre, einfach `seed + thread_id` als Seed zu nehmen. Das habe ich mit `seed_test.cpp` ausprobiert, das die ersten vier Zahlen für die Seeds 0, 1 und 2 ausgibt:

```
seed 0: 16807 282475249 1622650073 984943658
seed 1: 16807 282475249 1622650073 984943658
seed 2: 33614 564950498 1097816499 1969887316
```

Die Seeds 0 und 1 liefern also genau dieselben Zahlen. Mit `seed + thread_id` und `seed = 0` hätten Thread 0 und Thread 1 damit exakt dieselben Punkte erzeugt. Außerdem sind die Zahlen bei Seed 2 am Anfang genau doppelt so groß wie bei Seed 1, die Folgen hängen also stark zusammen. Das liegt daran, dass `default_random_engine` beim GCC ein sehr einfacher Generator ist, bei dem die nächste Zahl immer nur das Vorherige mal 16807 (modulo $2^{31} - 1$) ist. Der Seed 0 wird dabei intern zu 1 gemacht, weil sonst immer nur 0 herauskommen würde.

Deswegen baue ich den Seed mit `seed_seq` aus dem eigentlichen Seed und der Thread-ID zusammen. `seed_seq` mischt die beiden Werte, sodass jeder Thread einen ganz anderen Startwert bekommt.

### Meine parallele Version

Die Punkte werden wie bei Aufgabe 3 in Blöcken auf die Threads verteilt, jeder Thread erzeugt also $n$ geteilt durch die Threadanzahl Punkte. Für den Zähler benutze ich diesmal `reduction(+ : counter)` bei `#pragma omp parallel`. Dadurch bekommt jeder Thread automatisch seine eigene Kopie von `counter` und am Ende der parallelen Region werden alle Kopien zusammenaddiert. So braucht man weder `atomic` noch einen extra Vektor für die Teilergebnisse.

```
#pragma omp parallel reduction(+ : counter)
{
  ...
  seed_seq seq{seed, (unsigned int)id};
  default_random_engine re{seq};
  ...
}
```

### Ergebnisse

Mit $n = 100.000.000$ Punkten braucht die serielle Version von der Folie ohne Optimierung etwa 6,1 Sekunden und mit `-Ofast` etwa 1,6 Sekunden. Für die parallele Version kam Folgendes heraus (beste Zeit aus 3 Durchläufen):

| | | ohne Optimierung | | -Ofast | |
|---|---|---|---|---|---|
| Threads | $\pi$ | Zeit [s] | Speedup | Zeit [s] | Speedup |
| 1 | 3.14163648 | 6.1557 | 1.00 | 1.6054 | 1.00 |
| 2 | 3.14179564 | 3.2494 | 1.89 | 0.8226 | 1.95 |
| 4 | 3.14200724 | 1.6428 | 3.75 | 0.4485 | 3.58 |
| 8 | 3.14192952 | 1.2158 | 5.06 | 0.2855 | 5.62 |

Bis 4 Threads skaliert das Programm wieder fast linear und mit 8 Threads ist es noch einmal schneller, aber wie bei Aufgabe 3 nicht doppelt so schnell wie mit 4 Threads. Mit 8 Threads und `-Ofast` ist das Programm insgesamt etwa 21 mal schneller als die serielle Version ohne Optimierung.

Beim Ergebnis fällt auf, dass $\pi$ je nach Threadanzahl ein anderer Wert ist. Das ist hier aber normal, weil sich mit der Threadanzahl auch die Zufallszahlen ändern. Jeder Thread hat seinen eigenen Seed und erzeugt einen anderen Teil der Punkte. Bei gleicher Threadanzahl kommt dagegen immer genau dasselbe heraus, weil die Seeds fest sind. Insgesamt ist Monte Carlo deutlich ungenauer als die numerische Integration aus Aufgabe 3. Trotz 100 Millionen Punkten stimmt $\pi$ nur auf etwa 3 Nachkommastellen, weil der Fehler bei Monte Carlo nur mit $1/\sqrt{n}$ kleiner wird.